A Natural-Language Geospatial Query and Site-Evaluation System on PostGIS

Dr. Vaishali Wadhe
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
vwadhe@somaiya.edu

Riddhesh Chaudhary
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
riddhesh.c@somaiya.edu

Arpit Balmiki
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
arpit.b@somaiya.edu

Chirayu Vyas
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
chirayu.vyas@somaiya.edu

***Abstract*—Geospatial analysis supports urban planning, retail strategy and infrastructure decisions, yet traditional GIS platforms demand expertise in spatial query languages and specialised software. Large language models (LLMs) can lower this barrier, but systems that let the model answer directly tend to hallucinate place names and coordinates, which makes their output unverifiable. This paper presents GeoQuery, a Felt-style layered web map driven by a natural-language to PostGIS query engine grounded in open data. A user asks a spatial question in English, for example "hospitals within 1 km of a metro station". An LLM writes a PostGIS query against a documented schema, the query is executed on real data, and the system returns a GeoJSON answer layer, a grounded explanation and the SQL itself, so every answer carries provenance. On top of the engine we build intelligent decision tools: a click-to-evaluate site report, a competition and catchment analysis, a cost-to-open and break-even estimator, and a side-by-side comparison of up to three candidate locations. The system runs on PostgreSQL 16 with PostGIS 3.4, serves vector tiles with Martin, and renders them with MapLibre GL. It is loaded with Mumbai data from OpenStreetMap, a 76,000-row house-price dataset and area-level crime data, and its ETL is designed to generalise to other cities. We describe the architecture, the decision models and their current limitations, and report a first benchmark of query accuracy on 39 spatial questions.**

***Index Terms*—Generative AI, GIS, Natural Language Processing, Text-to-SQL, Spatial SQL, PostGIS, Site Selection**

I. Introduction

Geospatial data is central to urban planning, transportation, environmental monitoring and retail analytics. Geographic Information Systems (GIS) have traditionally been the main tools for such analysis, but they require users to understand spatial data handling, query languages such as SQL, and specialised software. This creates a barrier for the many people who have location questions but no GIS training.

Web platforms such as Felt have made map creation, collaboration and visualization much easier [1]. Even so, users must still translate a real-world question into manual operations on layers and filters. At the same time, LLMs have made natural-language interfaces to structured data practical, and recent work applies them to spatial data [2], [3].

A naive way to build such an interface is to let the LLM answer the question itself. Our first proof of concept (POC) did this and exposed three weaknesses. The model invented area names and coordinates. Ranking used a fixed hand-weighted formula over about 40 areas with synthetic 1 to 10 metrics. Map markers were placed with random coordinate jitter, and the maps were Leaflet raster maps with no real spatial computation. None of the outputs could be checked: the assistant answered with familiar area names, but nothing in the answer was computed from data.

The central idea of this work is to replace "the LLM hallucinates the answer" with "the LLM writes a structured spatial query that is executed on real data". Every output is then verifiable and reproducible, and the generated SQL is shown to the user as provenance.

GeoQuery implements this idea as a full-screen layered map with a natural-language query bar. Beyond exploratory queries, it supports decision-oriented work for prospective business owners: evaluating a clicked location, analysing local competition and catchment demand, estimating the cost to open a store, and comparing candidate sites. The main contributions are:

1) A natural-language to PostGIS query engine that grounds the LLM in a documented schema and returns a GeoJSON layer, an explanation and the executed SQL.
2) A real-data foundation: an ETL pipeline that loads OpenStreetMap, real-estate listings and crime data into a single spatial database, designed to work for any city.
3) A set of decision tools (site evaluation, competition analysis, cost-to-open, location comparison) built on the same data.

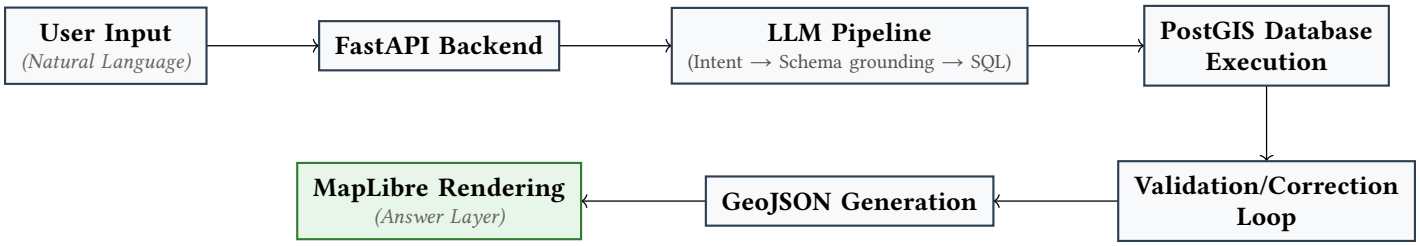


Fig. 1. System overview of GeoQuery: from a natural-language question to an answer layer on the map.

4) An honest account of what is and is not yet validated, together with the evaluation plan for the missing pieces.

II. Literature Survey

Traditional GIS platforms such as ArcGIS and QGIS provide buffering, proximity analysis and spatial querying, but assume knowledge of spatial concepts, data formats and query languages. Cloud platforms such as Felt and Mapbox improve collaboration and visualization [1], yet users still drive them through layers, filters and map controls.

Natural-language interfaces to databases map a question to an executable query. Benchmarks such as Spider [4] and BIRD [5] have driven progress on general text-to-SQL, with LLM-based systems now dominating. Spatial questions add difficulty: the model must pick the right geometry types, apply spatial predicates and distance functions correctly, and respect coordinate reference systems. Early geospatial interfaces used rule-based mapping from keywords such as "near" and "within" to spatial operations. These are effective for simple queries but brittle for ambiguous or compositional ones.

More recent work uses LLMs for geospatial tasks. A multi-agent spatial text-to-SQL pipeline reports that adding an execution-based review stage improves accuracy by roughly 11 points [3]. ChatGeoAI converts natural language into executable geospatial operations for non-expert users [2]. GeoLLM shows that LLMs combined with OpenStreetMap features can model quantities for which ground truth is scarce [6]. We take the first two as the design reference for our query engine. The 11-point figure is a result from that work, not from ours. We treat GeoLLM as relevant to estimating metrics, such as rent, where no direct data exists.

Spatial decision support systems extend GIS with multi-criteria decision analysis (MCDA) for tasks such as site selection [7]. They rank candidates by combining normalised criteria with weights, but typically require expert configuration and expert interpretation of the result.

OpenStreetMap (OSM) provides free, globally available data on roads, amenities and land use [8], and tools such as OSMnx make it easy to acquire polygon-bounded extracts for any city [9]. PostGIS adds spatial types, GiST indexing and spatial functions to PostgreSQL, which makes it a suitable execution target for generated queries.

Existing systems tend to cover either visualization, query processing or decision support. We combine all three on one open-data stack, with the emphasis on verifiable query results.

III. Proposed System

GeoQuery lets users ask questions about geospatial data in plain English and then act on the results with decision tools. Its workspace is a full-screen map with a layer panel, a natural-language query bar, and panels for evaluating, comparing and costing locations.

Fig. 1 gives an overview. A question goes from the browser to the FastAPI backend, where an LLM pipeline checks its intent, grounds it in the schema and generates a PostGIS query. The query is validated and executed, a correction loop retries it on an error or an empty result, and the rows are returned as a GeoJSON answer layer drawn by MapLibre GL. The stages are detailed in the Methodology section.

A. Problem Statement and Objectives

Geospatial analysis is essential for business planning, but existing GIS tools demand knowledge of spatial data and query languages and mostly leave users to interpret patterns by hand. A person deciding where to open a cafe must weigh prices, transit, safety, competition and start-up cost, and no single easy tool combines these. LLM interfaces lower the barrier, but when the model supplies the answer directly, place names and coordinates can be invented and nothing can be verified. A system is needed that accepts natural language, computes answers on real spatial data, shows how each answer was obtained, and supports practical location decisions.

The objective is therefore a natural-language geospatial query and decision-support system that accepts plain English, translates it into PostGIS queries through an LLM grounded in a documented schema, returns a map layer with an explanation and the executed SQL, and supports location decisions (site evaluation, competition and catchment analysis, cost-to-open and break-even estimation, and side-by-side comparison), using only open-source tools and open data.

B. Methodology

The system is a modular pipeline. A question enters through the query bar, the backend turns it into a spatial query, the database executes it, and the frontend draws the answer. Decision tools reuse the same database through dedicated endpoints.

a) *Explanation of Methodology:*

1) **User input.** The user types a spatial question, either exploratory ("hospitals within 1 km of a metro station") or decision-oriented.
2) **Intent check.** A first LLM call decides whether the question is a clear spatial query and labels its task type (filter, proximity, ranking, aggregation, site selection or other). If the target is genuinely unknowable, for example "best place?", the system returns a short follow-up question instead of running a query.
3) **Schema grounding.** The SQL-generating call receives a compact description of the queryable schema in its prompt: the tables and columns of the admin, roads, transport, poi, realestate, demo, crime and env schemas, their geometry types and the allowed values of key categorical columns, plus rules to filter every table by city_id, use only listed tables and columns, and emit one read-only SELECT with a geometry column named geom. Because the model can only reference data that exists, it cannot invent places. The same description is stored as a 34-row meta.schema_catalog table, which the engine does not read at query time.
4) **Query generation and validation.** The LLM (Groq openai/gpt-oss-120b) writes a PostGIS query using functions such as ST_DWithin and ST_Contains over EPSG:4326 geometry indexed with GiST. Before execution the text must be a single SELECT or WITH statement with no chained statements and no data-modifying or DDL keywords.
5) **Execution and repair.** The query runs on PostgreSQL 16 with PostGIS 3.4 as a dedicated geo_readonly role, with a 15 second statement timeout and a cap of 5,000 features. If it fails validation or raises a database error, the error text is fed back to the model for a corrected query (at most two repairs), and if a valid query returns no rows the model is asked once to loosen it. The best successful attempt is kept, preferring a non-empty one, and every attempt is returned to the client. This is a simplified form of the execution-based review in [3], with at most four attempts in total.
6) **Result and visualization.** Rows become a GeoJSON answer layer, and the LLM writes a short explanation grounded in a sample of the returned rows; the generated SQL is returned alongside, so users can inspect how the answer was obtained. MapLibre draws the layer next to the base layers, which Martin serves as Mapbox Vector Tiles directly from PostGIS.

C. Decision Support Models

The decision tools reuse the same database through separate endpoints.

Site evaluation. For a clicked point and a radius of 0.5 to 3 km, the report gives the median price per sqft of nearby listings, a typical price band, a comparison with the city median, a breakdown by size band, the nearest metro, rail and bus stops, amenity counts (healthcare, education, food, banking, shopping), and a safety score with a risk level.

Competition and catchment analysis. The report gives the competitor count, density (per km²), nearest competitor, a saturation label (below 5 low, 5 to 15 moderate, 15 to 30 high, above 30 saturated) and a 0 to 100 catchment demand index. Competitors are points in poi.place whose category matches the business type inside the radius. The demand index counts points of interest in five groups (healthcare, education, food, banking and shopping) and transit stops, caps each count (40, 25, 40, 15, 30 and 12 respectively) so that dense areas do not dominate, and combines them with business-specific weights. To suggest gaps, a 5 by 5 grid of candidate spots covers the radius, each scored as its demand index divided by one plus its competitors within 500 m; up to three spots with a demand index of at least 15 and a score at least 15% above the evaluated spot are suggested (green markers; competitors are red dots).

Cost-to-open and break-even. The estimator takes a store area in sqft and returns one-time setup costs, monthly running costs and working capital. Because rental data is not available, rent is estimated from sale prices: the median sale price per sqft of listings within 2.5 km times 0.5% (a rent-to-value ratio) and a commercial premium of 1.15, or 120 rupees per sqft per month if no listing is nearby. The security deposit is six months of rent and working capital is three months of running costs; other line items come from the LLM, with a heuristic fallback. Customers per day are the store area (per 100 sqft) times a benchmark rate for the business type, a footfall multiplier of 0.6 to 1.5 (amenities within 1 km, metro or rail within 800 m) and a competition factor $\frac{1}{1+0.04n}$ floored at 0.6, where n is the number of competitors within 1 km. Revenue is customers times an average ticket (scaled by 0.8, 1.0 or 1.3 by tier) times 30 days, and profit is revenue minus cost of goods (a fixed share of revenue) and the remaining monthly costs. The conservative, base and optimistic scenarios scale customers by 0.7, 1.0 and 1.3. Payback is the setup cost divided by monthly profit, and break-even revenue is the remaining monthly cost divided by one minus the cost-of-goods share. The per-type benchmarks are assumptions set by the authors, not measured values, and because the LLM line items vary, two runs for the same site can give different totals, profit and payback.

Location comparison. Up to three pinned spots (A, B, C) are shown side by side across price, access, safety, competitors, rent, start-up cost, revenue, profit and payback. The best value in each row is marked.

Area-level weighted ranking (legacy dashboard). The original dashboard ranks areas with a standard weighted-sum MCDA model [7], $S_i = \sum_k w_k m'_{ik}$ with $\sum_k w_k = 1$, over footfall, youth, access, rent and competition scored 0 to 10 (rent and competition inverted as $10 - m_{ik}$), with hand-picked weights 0.30, 0.25, 0.20, 0.15 and 0.10 in that order. In the POC the metrics were synthetic.

To test how much the ranking depends on these weights we ran a sensitivity study on the 40 scored areas (script in the eval/ folder). We compared the hand-picked weights with equal, rank-order centroid (ROC), AHP (consistency ratio 0.011), entropy and CRITIC weights, three scenario weight sets (budget, student and premium) and TOPSIS, measured

agreement by Kendall's τ and by the overlap of the top five areas, perturbed each weight by up to 30% one at a time, and drew 1,000 random weight vectors (seed 42). The ranking is stable under small changes to a single weight (worst-case τ of 0.90 or higher for a 30% change) but not under a change of scheme: AHP, ROC and the premium scenario keep the baseline's top five (τ of 0.92, 0.81 and 0.83), while the other schemes share only 0 to 2 of its top five areas (τ from −0.11 to 0.87). Across random weight vectors the median τ is 0.33 (5th to 95th percentile −0.49 to 0.84), and no area stays in the top five in more than 45% of draws. The weights therefore encode a preference, not a neutral measurement, which is one reason the GeoQuery workspace uses measured spatial queries in place of this fixed ranking. These results describe the 40-area scored dataset, not real-world site quality.

D. Implementation

GeoQuery has three layers: a presentation layer (React 19, Zustand and MapLibre GL), an application layer (FastAPI with the LLM client, and MongoDB for chat history only) and a data layer (PostgreSQL 16 with PostGIS 3.4, and Martin for vector tiles); Table I lists the components. This separation keeps the model, the query execution and the rendering independently replaceable. The database holds all layers in EPSG:4326 with GiST indexes and a city_id on every row, so several cities can coexist. The ETL (etl/) ingests roads, transit, POIs and boundaries from OpenStreetMap by city polygon (OSMnx), geocodes 228 real-estate localities with Nominatim and loads the crime data, so the same pipeline can load other cities; the deployed instance is pre-loaded for Mumbai. In the workspace the left panel toggles vector layers (administrative boundaries, roads, transit stops, businesses and POIs, real-estate listings, and a crime and safety score), each with an opacity slider. Users can click a feature to inspect it, type a question into the query bar, or click a point to open the evaluation panel with its Competition and Cost-to-open tabs. The backend exposes /nlquery, /site-eval, /competitor-analysis, /business-estimate and /health. The original assistant dashboard (/dashboard, a chat assistant with a Leaflet heatmap that ranks areas) is kept as the baseline. The system runs on a standard laptop (Intel i5, 8 GB RAM, 20 to 50 GB of storage) using open-source software and open data throughout.

TABLE I
System component stack.

Presentation	React 19, MapLibre GL, Zustand; layer panel, query bar, evaluation panel
Application	FastAPI endpoints, LLM client (groq_client.py), MongoDB chat history
Tiles	Martin serving MVT from PostGIS
Data	PostgreSQL 16 + PostGIS 3.4 + pgvector; schemas admin, roads, transport, poi, realestate, demo, crime, env; meta.schema_catalog; core.city
ETL	OSMnx ingest, Nominatim geocoding, crime data migration

E. Limitations

1) **Small benchmark.** The NL to SQL benchmark has 39 questions in six categories and was run once, so its accuracy figures carry substantial uncertainty. No query needed a retry, so the benefit of the repair stage is unmeasured.
2) **Basic SQL safeguards.** Generated SQL runs under a read-only role with a statement timeout and a row cap and is checked to be a single SELECT. The check is keyword-based and not a full SQL parser, and there is no table allow-list beyond the role's grants. The LLM is called without a fixed temperature, so the same question can yield different SQL.
3) **Modelled business figures.** There is no real rental data, so rent is estimated from sale prices. The saturation bands, demand weights and per-type revenue benchmarks are set by judgement and not calibrated against real outcomes. The non-rent cost items and the report summary are written by the LLM and not reconciled with the computed totals, so totals, profit and payback can differ between runs for the same site and the summary can disagree with the figures beside it.
4) **Coarse spatial data.** Real-estate listings are geocoded at locality level (228 localities), catchments are circles and not travel-time isochrones, crime data is area-level, and OSM completeness varies.
5) **Coverage and usability.** Only Mumbai is loaded. Pinned locations are not saved, and the interface is designed for desktop.

IV. Results And Discussion

A. Implemented System

The system is implemented end to end. Table II lists the Mumbai data currently loaded.

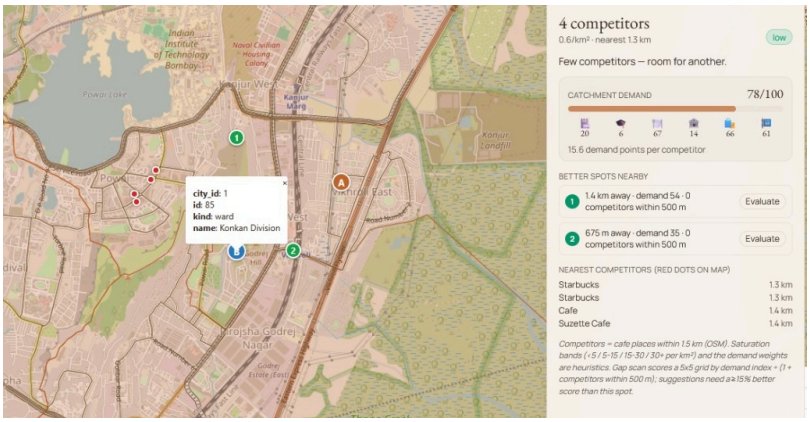
TABLE II
Mumbai data loaded in the current instance.

Dataset	Size
Administrative boundaries	48
Road segments	16.8k
Transit stops	2.2k
Businesses and POIs	10.8k
Real-estate listings (CSV)	76,000 rows
Geocoded real-estate localities	228
Schema catalog entries	34

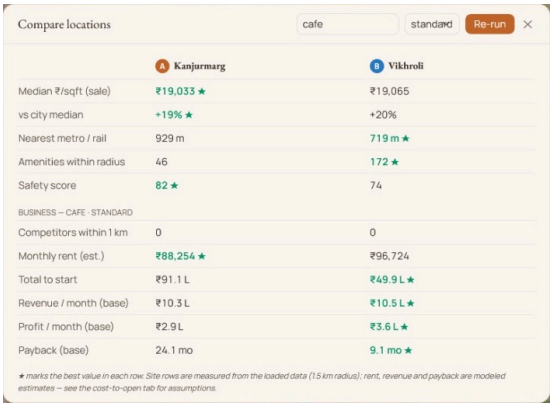
Compared with the POC, the GeoQuery workspace replaces invented coordinates with coordinates from the database, replaces random marker jitter with real geometries, replaces raster Leaflet maps with vector tiles, and replaces fixed synthetic metrics with metrics computed from nearby listings, transit and amenities.

B. Qualitative Demonstration

A query such as "hospitals within 1 km of a metro station" is turned into a spatial SQL query, executed and



(a) Competition tab for a cafe at Vikhroli (B), 1.5 km radius: competitors in red, suggested spots in green.



(b) Comparison of Kanjurmarg (A) and Vikhroli (B), cafe, standard tier.

Fig. 2. Decision tools in the GeoQuery workspace. Site rows in (b) are measured within 1.5 km; rent, start-up cost, revenue, profit and payback are modelled estimates from a single run.

drawn as an answer layer, with the explanation and the SQL in the panel. Clicking a location opens a site report, and the Competition and Cost-to-open tabs extend it to a business decision. Fig. 2 shows this for a cafe at Vikhroli (B), compared with Kanjurmarg (A): the competition tab finds four cafes within 1.5 km, the nearest 1.3 km away, so neither site has a competitor within 1 km, and suggests two lower-competition spots (green). The comparison table puts measured site attributes (price, transit, amenities, safety) next to the modelled business figures.

C. Comparative Analysis

We compare the system along two axes: the query engine against its own ablations and against the direct-LLM approach of the POC, and the weighting schemes of the legacy area ranking.

a) *Query engine versus ablations and the direct-LLM baseline:*

The benchmark in eval/nl2sql/ has 39 spatial questions in six categories (filters, proximity, named areas, rankings, listings and compositional queries), each with a hand-written gold SQL query. A predicted layer is scored by comparing its geometry set with the executed gold result (exact match and F1), so column names do not matter. Three configurations are compared: the full engine; the same engine restricted to its first generation attempt (the first SQL of each full run, re-executed, which makes a paired comparison); and a direct-LLM baseline in which the model names places and coordinates without a database. The baseline returns no SQL, so it is scored on grounding: the share of returned names that exist in the database and the share of returned points near a real feature. Results are from a single run with openai/gpt-oss-120b and default sampling; the intent-check and explanation calls were skipped because they do not change the SQL.

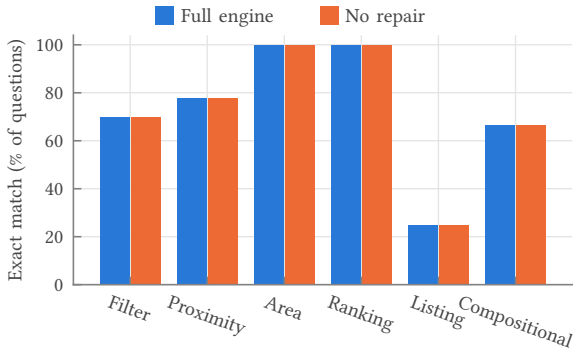
TABLE III
BENCHMARK RESULTS ON 39 QUESTIONS, SINGLE RUN.

Configuration	Exact	Geom. F1	Executed
Full engine	74.4%	88.4%	100%
First attempt only (no repair)	74.4%	88.4%	100%

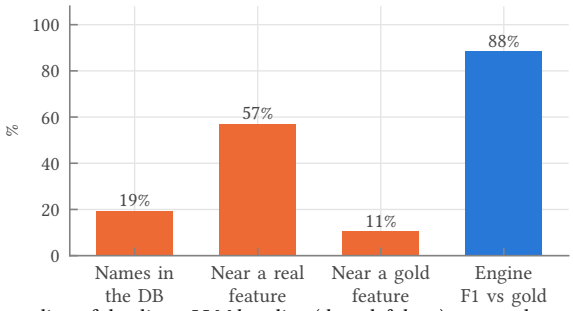
The full engine returns exactly the gold layer for 29 of the 39 questions (74.4%, geometry F1 88.4%), and every generated query executed and returned rows (Table III, Fig. 3(a)). The repair stage was never triggered, so the first-attempt-only configuration scores identically and this benchmark shows no benefit from repair. Accuracy is highest for named-area and ranking questions (both 100%) and lowest for listings (25%). Eight of the ten failures come from a row cap and not from a wrong query: the schema notes tell the model to use LIMIT 500, so questions whose gold answer is larger than 500 rows are truncated, and joined queries without DISTINCT let duplicate rows use up the limit. The other two are over-inclusive category choices (universities matched together with colleges, and motorways with a pattern broader than the gold road class). We count all ten as failures, but they point to the prompt rules and to the exact-match scoring more than to a lack of spatial reasoning.

The direct-LLM baseline gives a very different picture (Fig. 3(b)). Only 19.5% of the place names it returned exist in the database, 57.0% of its coordinates lie within 150 m of some real feature, and only 10.6% lie within 200 m of a feature in the gold answer. The coordinate figure is generous, because Mumbai is dense and a random point is often near some point of interest, so the other two figures are more informative. The baseline returns no SQL, so none of its answers can be checked. The median latency of the engine was 4.5 s, with a 95th percentile of 13.8 s (one LLM call plus the database, without the intent-check and explanation stages), against 13.2 s for the baseline. With 39 questions and one run, differences of a few questions should not be over-interpreted. A further ablation, in which the schema description lists only table names, is implemented but was not run.

b) *Weighting schemes of the legacy ranking:*

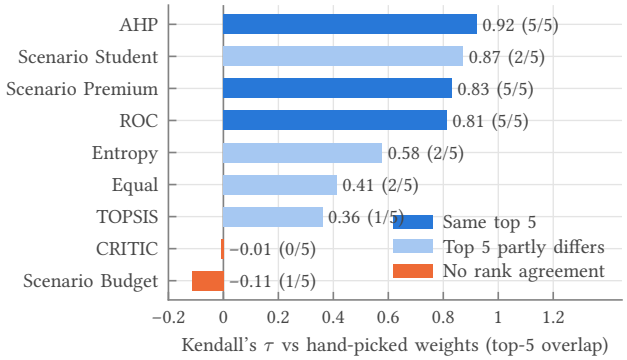


(a) Exact-match accuracy by question category.

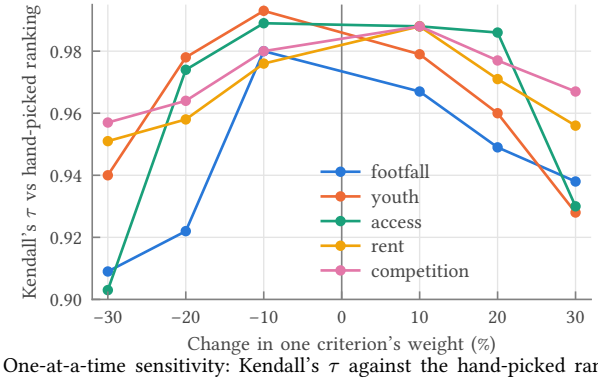


(b) Grounding of the direct-LLM baseline (three left bars) next to the engine's geometry F1 against the gold answer (right bar).

Fig. 3. Benchmark results. The first-attempt-only configuration in (a) is identical to the full engine because no repair was needed. The metrics in (b) differ: the baseline bars show how much of its output is real, the engine bar its agreement with the gold layer.



(a) Rank agreement (Kendall's τ) of each weighting scheme with the hand-picked weights, and the overlap of the top-five areas.



(b) One-at-a-time sensitivity: Kendall's τ against the hand-picked ranking when one criterion's weight is changed by up to 30%, the others being renormalised.

Fig. 4. Sensitivity of the legacy area ranking to its weights.

The comparison of weighting schemes is described with the decision support models above. Fig. 4(a) shows how far each scheme's ranking is from the hand-picked one, and Fig. 4(b) shows how little the ranking changes when a single weight is changed by up to 30%.

D. Discussion

Grounding the model in a documented schema and returning the executed SQL changes the nature of the interface: answers can be checked, reproduced and corrected, and the decision tools show that the same data supports practical questions, such as comparing three candidate shop locations on price, access, safety and competition.

The main weakness is evaluation. The benchmark is small, was run once and did not exercise the repair stage, so we cannot say how much repair helps or how the engine behaves on harder questions, and the safeguards (a read-only role, a statement timeout, a row cap and a single-statement check) have not been tested against adversarial input. Cost and revenue estimates inherit the approximations in the rent model, the benchmarks and the LLM line items, so the measured site attributes are more reliable than the modelled business figures and the estimates should be treated as planning aids. The weight study shows that the legacy area ranking is sensitive to the choice of weighting scheme.

V. Conclusion And Future Scope

A. Conclusion

GeoQuery shows that a natural-language interface to geospatial data can be built so that answers are computed on real data and are inspectable. By pairing a schema-grounded LLM with PostGIS, vector tiles and open data, it avoids the invented places and coordinates of direct LLM answers, and it extends the query engine with site evaluation, competition analysis, cost-to-open modelling and location comparison for practical decisions. The system is functional for Mumbai, and its ETL is designed to generalise. A first benchmark of 39 questions shows 74.4% exact-match accuracy against hand-written gold queries, with most failures caused by a row cap, whereas a direct-LLM baseline returns real place names only 19.5% of the time. A larger benchmark remains to be built.

B. Future Scope

1) **Query reliability:** extend the benchmark (more questions, repeated runs, other models and harder queries that exercise the repair stage) in the style of Spider and BIRD [3], [4], [5], remove the row-cap and over-inclusion failures found here, and replace the keyword check with parser-based validation and a table allow-list.

2) **Data and models:** integrate real rental data, calibrate the saturation bands and revenue benchmarks against real outcomes, fix or cache the LLM cost items per site so that repeated runs agree, and extend the metrics modelling with LLM and OSM features where ground truth is scarce [6].
3) **Coverage and usability:** replace radius catchments with isochrones, load additional cities using the existing ETL, and add saved sites and a mobile layout.

References

[1] Felt, "Felt AI: AI-powered GIS and mapping platform." [Online]. Available: https://help.felt.com/felt-ai

[2] A. Mansourian and R. Oucheikh, "ChatGeoAI: Enabling Geospatial Analysis for Public through Natural Language, with Large Language Models," *ISPRS International Journal of Geo-Information*, vol. 13, no. 10, p. 348, 2024.

[3] A. K. Kazazi, Z. Li, M. N. Lessani, and G. Cervone, "From Questions to Queries: An AI-powered Multi-Agent Framework for Spatial Text-to-SQL," *arXiv preprint arXiv:2510.21045*, 2025.

[4] T. Yu *et al.*, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 3911–3921.

[5] J. Li *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] R. Manvi, S. Khanna, G. Mai, M. Burke, D. Lobell, and S. Ermon, "GeoLLM: Extracting Geospatial Knowledge from Large Language Models," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] J. Malczewski, *GIS and Multicriteria Decision Analysis*. New York: John Wiley & Sons, 1999.

[8] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[9] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.